LOGIC GATES

Complete Xilinx Vivado Design Flow

LAB EXPERIMENT

AND · OR · NOT · XOR · XNOR

RTL Design → Testbench → Simulation → Synthesis → Implementation → FPGA Verification

Hardware: Digilent Basys 3 | FPGA: Xilinx Artix-7 (XC7A35T)

Submitted By

KUKKALA BHAVYA SREE

Under Guidance of

Ms. Harika

1. Project Overview

This document presents the complete step-by-step Xilinx Vivado design flow for implementing five fundamental logic gates on a Digilent Basys 3 FPGA development board. The flow covers every stage from project creation to physical hardware verification.

Stage	Tool / Action	Purpose
1	Create Vivado Project	Set up workspace, target device (Artix-7)
2	Write RTL Verilog	Behavioral description of 5 logic gates
3	Write Testbench	Functional simulation stimulus
4	Run Simulation	Verify logic correctness in XSim
5	Synthesis	Map Verilog to FPGA primitives (LUTs)
6	Implementation	Place & Route on Artix-7 fabric
7	Generate Bitstream	Create programming file (.bit)
8	Program FPGA	Download bitstream to Basys 3
9	Hardware Verification	Test all 4 input combinations (00,01,10,11)

1.1 Target Hardware

Component	Specification
Board	Digilent Basys 3
FPGA	Xilinx Artix-7 XC7A35T-1CPG236C
Logic Cells	33,280
I/O Standard	LVCMOS33 (3.3 V)
Inputs Used	2 slide switches (SW0, SW1) → a, b
Outputs Used	5 LEDs (LD0–LD4) → y_and, y_or, y_not, y_xor, y_xnor
Clock	Not used (combinational design)
Tool	Xilinx Vivado Design Suite 2023.x

2. Step 1 — Create Vivado Project

2.1 Launch Vivado & New Project Wizard

STEP 1

Open Vivado
Launch Xilinx Vivado. On the welcome screen, click 'Create Project'.

STEP 2

Project Name & Location
Project name: Logic_Gates Project location: C:/Users/bhanu/Logic_Gates Check 'Create project subdirectory' → Click Next

STEP 3

Project Type
Select 'RTL Project' Check 'Do not specify sources at this time' → Click Next

STEP 4

Select Target Device
Family: Artix-7 Package: cpg236 Speed Grade: -1 Select: xc7a35tcpg236-1 (Basys 3 device) → Click Next → Finish

2.2 Vivado Project Settings Summary

Setting	Value
Project Name	Logic_Gates
Project Location	C:/Users/bhanu/Logic_Gates/
Project Type	RTL Project
Top Module	Logicgates
Target Device	xc7a35tcpg236-1
FPGA Family	Artix-7
Simulator	Vivado Simulator (XSim)
Synthesis Tool	Vivado Synthesis

3. Step 2 — Write RTL Verilog Design

3.1 Add Source File

STEP 5	**Create Verilog Source** In the Sources panel → click '+' → Add or Create Design Sources Click 'Create File' → File type: Verilog → Name: Logicgates → OK → Finish

STEP 6	**Write the RTL Code** Double-click Logicgates.v in Sources panel and enter the code below:

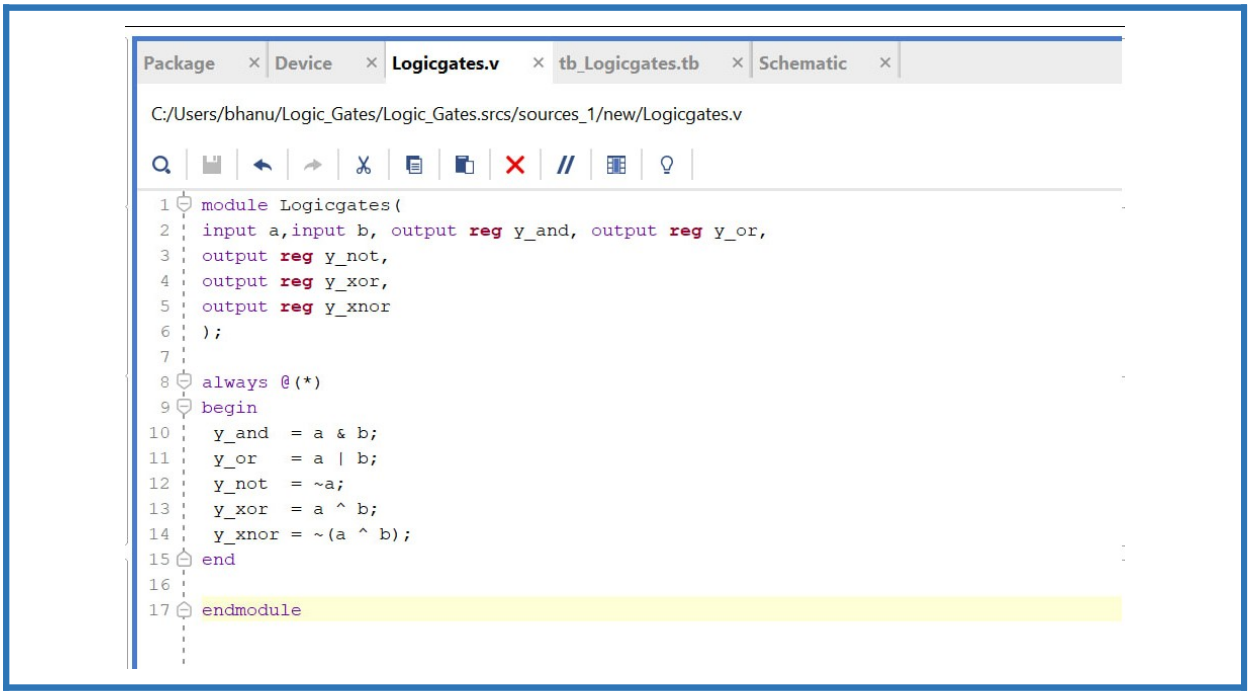


Figure 1: Logicgates.v — RTL Source Code in Vivado Editor

3.2 Complete Verilog Source Code

```
module Logicgates(
  input  a, input b,
  output reg y_and,
  output reg y_or,
  output reg y_not,
  output reg y_xor,
  output reg y_xnor
);
```

```
always @(*) begin
  y_and  = a & b;        // AND gate
  y_or   = a | b;        // OR gate
  y_not  = ~a;           // NOT gate (inverts only input a)
  y_xor  = a ^ b;        // XOR gate
  y_xnor = ~(a ^ b);     // XNOR gate
end
endmodule
```

3.3 Design Explanation

Line	Statement	Gate	Logic Description
10	y_and = a & b	AND	Output HIGH only when BOTH a=1 AND b=1
11	y_or = a | b	OR	Output HIGH when a=1 OR b=1 (or both)
12	y_not = ~a	NOT	Output is complement of a (b is ignored)
13	y_xor = a ^ b	XOR	Output HIGH when a and b are DIFFERENT
14	y_xnor = ~(a ^ b)	XNOR	Output HIGH when a and b are SAME

4. Step 3 — Write Testbench

4.1 Add Simulation Source

STEP 7	**Create Testbench File** Sources panel → '+' → Add or Create Simulation Sources Click 'Create File' → File type: Verilog → Name: tb_Logicgates → OK → Finish

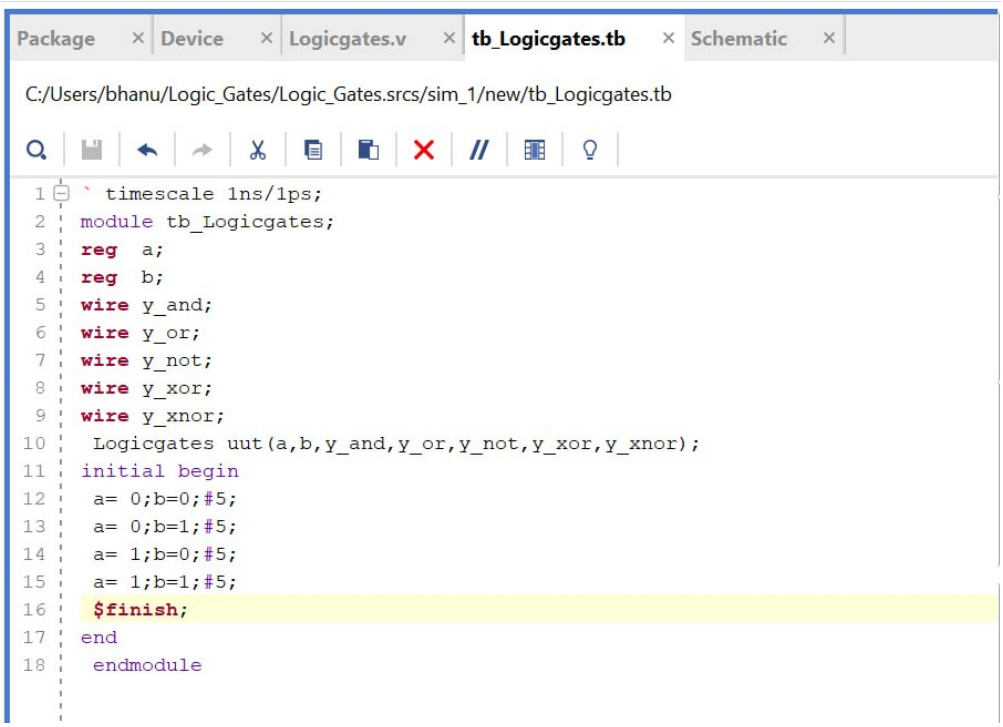


Figure 2: tb_Logicgates.tb — Testbench Source Code in Vivado

4.2 Testbench Code

```
`timescale 1ns/1ps
module tb_Logicgates;
  reg  a, b;
  wire y_and, y_or, y_not, y_xor, y_xnor;

  // Instantiate Design Under Test
  Logicgates uut (a, b, y_and, y_or, y_not, y_xor, y_xnor);

  initial begin
    a=0; b=0; #5;   // Test case 1: Input = 00
    a=0; b=1; #5;   // Test case 2: Input = 01
    a=1; b=0; #5;   // Test case 3: Input = 10
    a=1; b=1; #5;   // Test case 4: Input = 11
    $finish;        // End simulation
  end
endmodule
```

Test Case	a	b	Expected y_and	Expected y_or	Expected y_not	Expected y_xor	Expected y_xnor
Case 1	0	0	0	0	1	0	1
Case 2	0	1	0	1	1	1	0
Case 3	1	0	0	1	0	1	0
Case 4	1	1	1	1	0	0	1

5. Step 4 — Run Functional Simulation

5.1 Launch XSim Simulation

STEP 8

Set Testbench as Top

In Sources panel → Simulation Sources → right-click tb_Logicgates → 'Set as Top'

STEP 9

Run Behavioral Simulation

Flow Navigator (left panel) → Simulation → 'Run Behavioral Simulation' Vivado launches XSim and opens the waveform viewer automatically

STEP 10

Add Signals to Waveform

In the Scope panel, select tb_Logicgates In the Objects panel, select all signals → right-click → 'Add to Wave Window' Click the Run button (▶️) to run the simulation

STEP 11

Zoom to Fit

Press Ctrl+Shift+F or click 'Zoom Fit' button to view all waveforms

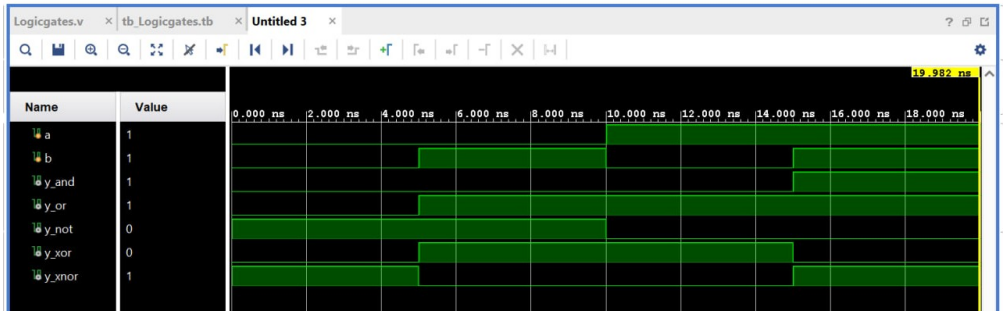


Figure 3: XSim Waveform — All 5 Gate Outputs Verified Across 4 Input Combinations

5.2 Waveform Analysis

Time (ns)	a	b	y_and	y_or	y_not	y_xor	y_xnor	Status
0–5	0	0	0	0	1	0	1	PASS
5–10	0	1	0	1	1	1	0	PASS
10–15	1	0	0	1	0	1	0	PASS
15–20	1	1	1	1	0	0	1	PASS

All 4 test cases PASS. The simulation confirms correct logic behavior for all five gates. Proceed to synthesis.

6. Step 5 — Synthesis

Synthesis translates the behavioral Verilog code into a gate-level netlist using FPGA primitives (LUTs, I/O buffers). Vivado performs logic optimization during this stage.

STEP 12

Run Synthesis

Flow Navigator → Synthesis → 'Run Synthesis' Vivado will synthesize the design (typically 1–2 minutes for this small design). Click OK on any dialogs that appear.

STEP 13

View Synthesis Reports

After synthesis completes → click 'Open Synthesized Design' Go to Reports → Synthesis → Utilization Report to check resource usage

STEP 14

View RTL Schematic

Open Synthesized Design → menu: Tools → Schematic Or: Flow Navigator → RTL Analysis → 'Open Elaborated Design' → Schematic

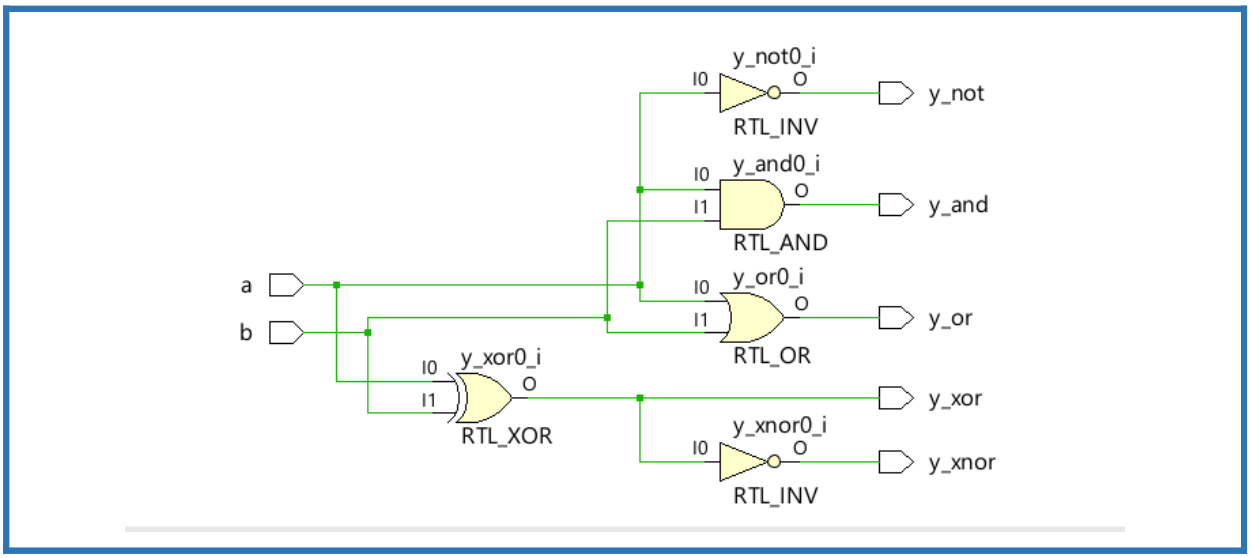


Figure 4a: RTL Analysis Schematic — Elaborated Design showing RTL_INV, RTL_AND, RTL_OR, RTL_XOR gate symbols

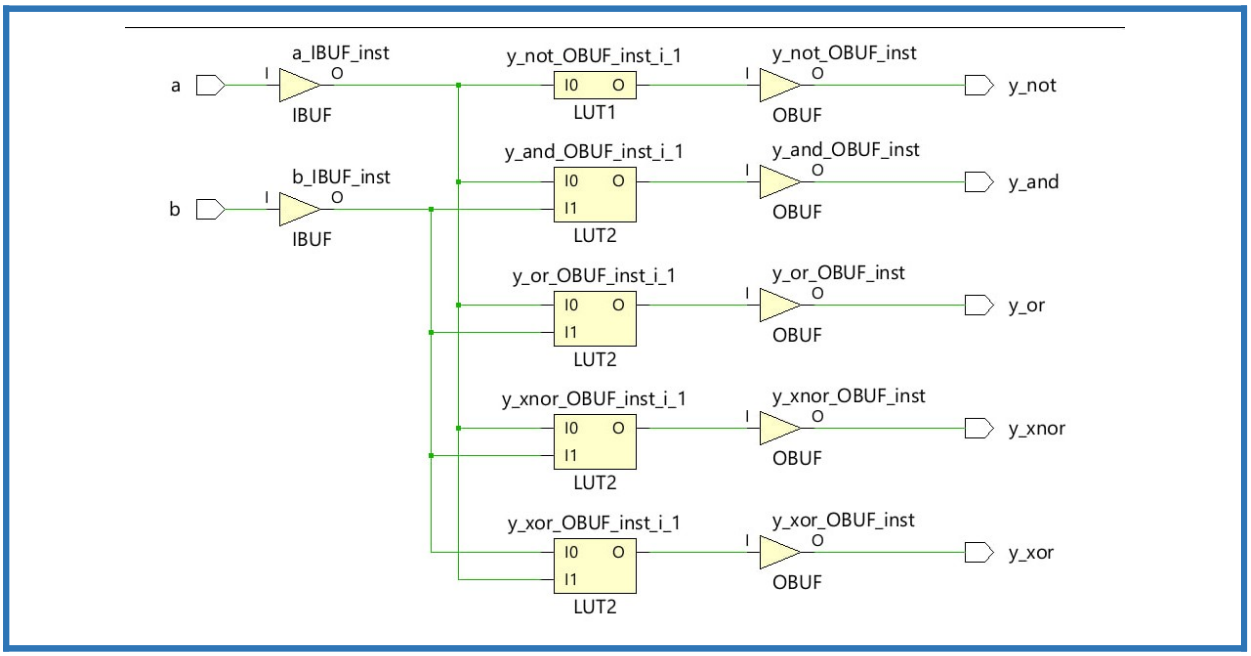


Figure 4b: Synthesized Netlist Schematic — Mapped to IBUF, LUT1, LUT2, OBUF FPGA Primitives

6.1 Synthesis Results

Primitive	Count	Maps To	Notes
IBUF	2	Inputs a, b	Auto-inserted I/O buffers for input pins
LUT1	1	y_not	1-input LUT — NOT gate (only uses input a)
LUT2	4	y_and, y_or, y_xor, y_xnor	2-input LUTs — one for each gate
OBUF	5	All 5 outputs	Auto-inserted output drive buffers

7. Step 6 — Add Pin Constraints (XDC)

The XDC (Xilinx Design Constraints) file maps logical design ports to physical Basys 3 pins — switches for inputs and LEDs for outputs.

STEP 15

Create Constraints File

Sources panel → '+' → Add or Create Constraints Click 'Create File' → Name: constraints → OK → Finish

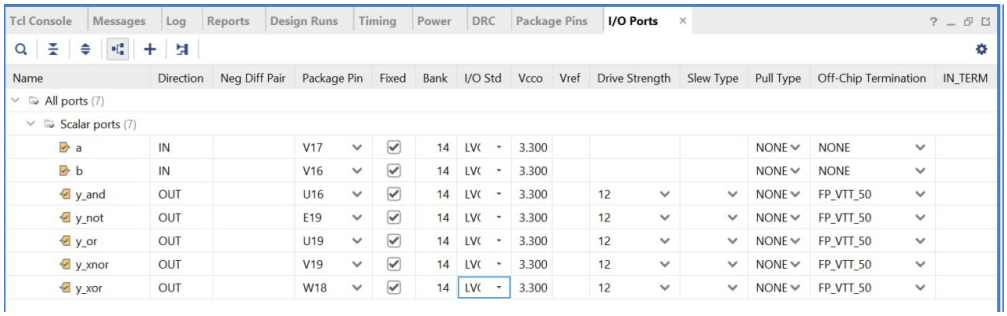
STEP 16

Enter Pin Constraints

Double-click constraints.xdc and enter the following:

```
## Inputs — Slide Switches
set_property PACKAGE_PIN V17  [get_ports {a}]
set_property IOSTANDARD LVCMOS33 [get_ports {a}]
set_property PACKAGE_PIN V16  [get_ports {b}]
set_property IOSTANDARD LVCMOS33 [get_ports {b}]
```

```
## Outputs — LEDs
set_property PACKAGE_PIN U16  [get_ports {y_and}]
set_property IOSTANDARD LVCMOS33 [get_ports {y_and}]
set_property PACKAGE_PIN E19  [get_ports {y_not}]
set_property IOSTANDARD LVCMOS33 [get_ports {y_not}]
set_property PACKAGE_PIN U19  [get_ports {y_or}]
set_property IOSTANDARD LVCMOS33 [get_ports {y_or}]
set_property PACKAGE_PIN V19  [get_ports {y_xnor}]
set_property IOSTANDARD LVCMOS33 [get_ports {y_xnor}]
set_property PACKAGE_PIN W18  [get_ports {y_xor}]
set_property IOSTANDARD LVCMOS33 [get_ports {y_xor}]
```



Tcl Console | Messages | Log | Reports | Design Runs | Timing | Power | DRC | Package Pins | I/O Ports

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
All ports (7)													
Scalar ports (7)													
a	IN		V17	✓	14	LV(	3.300				NONE	NONE	
b	IN		V16	✓	14	LV(	3.300				NONE	NONE	
y_and	OUT		U16	✓	14	LV(	3.300		12		NONE	FP_VTT_50	
y_not	OUT		E19	✓	14	LV(	3.300		12		NONE	FP_VTT_50	
y_or	OUT		U19	✓	14	LV(	3.300		12		NONE	FP_VTT_50	
y_xnor	OUT		V19	✓	14	LV(	3.300		12		NONE	FP_VTT_50	
y_xor	OUT		W18	✓	14	LV(	3.300		12		NONE	FP_VTT_50	

Figure 5: I/O Ports Panel — Physical Pin Assignments in Vivado

Port	Direction	Physical Pin	Basys 3 Component	I/O Standard
a	Input	V17	Slide Switch SW0	LVCMOS33
b	Input	V16	Slide Switch SW1	LVCMOS33
y_and	Output	U16	LED LD0	LVCMOS33
y_not	Output	E19	LED LD1	LVCMOS33
y_or	Output	U19	LED LD2	LVCMOS33
y_xnor	Output	V19	LED LD3	LVCMOS33
y_xor	Output	W18	LED LD4	LVCMOS33

8. Step 7 — Implementation (Place & Route)

Implementation places the synthesized netlist onto specific FPGA resources and routes the signal connections between them on the Artix-7 die.

STEP 17

Run Implementation

Flow Navigator → Implementation → 'Run Implementation' Click OK on any dialogs. This takes 1–3 minutes for this design.

STEP 18

Open Implemented Design

After completion → click 'Open Implemented Design' Review the Device view to see placed logic and routed nets on the Artix-7

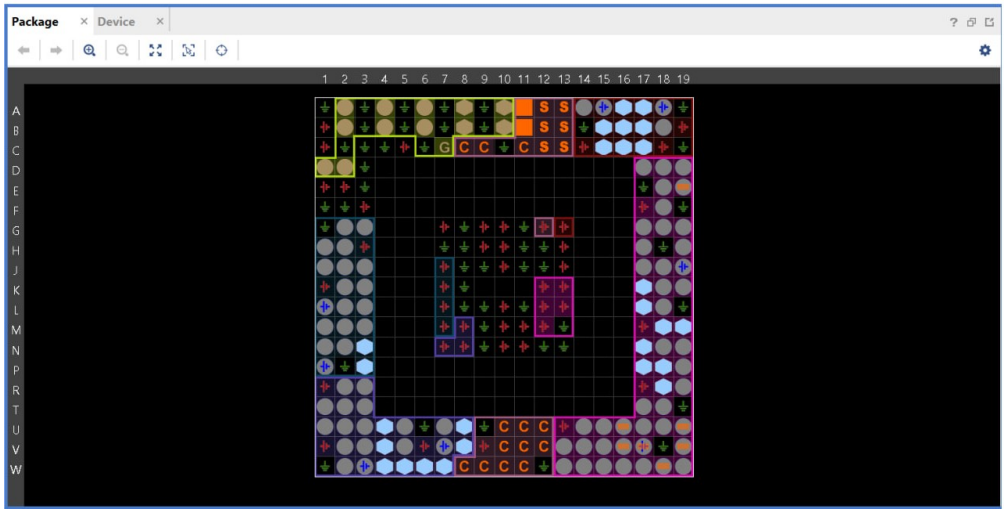


Figure 6: FPGA Package View — Physical Die Layout After Place & Route

8.1 Implementation Results

Report	Key Finding
Timing	No timing constraints needed — combinational design with no clock
Utilization	5 LUTs used — <1% of 20,800 available on XC7A35T
DRC	No critical warnings — design is clean
Power	Total: 1.63 W | Dynamic: 1.556 W | Static: 0.074 W
Temperature	Junction: 33.1°C | Thermal margin: 51.9°C

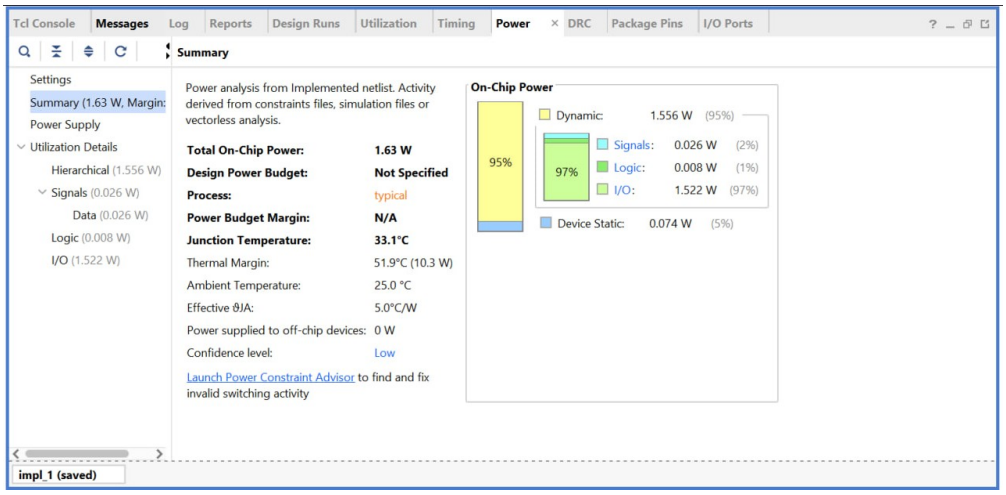


Figure 7: Power Analysis Summary — On-Chip Power Breakdown After Implementation

9. Step 8 — Generate & Program Bitstream

STEP 19	**Generate Bitstream** Flow Navigator → Program and Debug → 'Generate Bitstream' Vivado runs synthesis + implementation + bitstream generation in one step if not already done. Time: ~3–5 minutes total.
STEP 20	**Connect Basys 3 Board** Connect Basys 3 to your PC via USB (micro-USB cable to PROG port) Power ON the board using the power switch Windows should auto-install Digilent USB drivers
STEP 21	**Open Hardware Manager** After bitstream generation → click 'Open Hardware Manager' Click 'Open Target' → 'Auto Connect' Vivado detects the Basys 3 board automatically
STEP 22	**Program the Device** In Hardware Manager → right-click xc7a35t → 'Program Device' Bitstream file: Logic_Gates/Logic_Gates.runs/impl_1/Logicgates.bit Click 'Program' — takes ~5 seconds DONE LED on Basys 3 will light up green

10. Step 9 — Hardware Verification on Basys 3

After programming, all four input combinations (00, 01, 10, 11) are tested using the slide switches. The LED outputs are observed and verified against the expected truth table.

10.1 Test Procedure

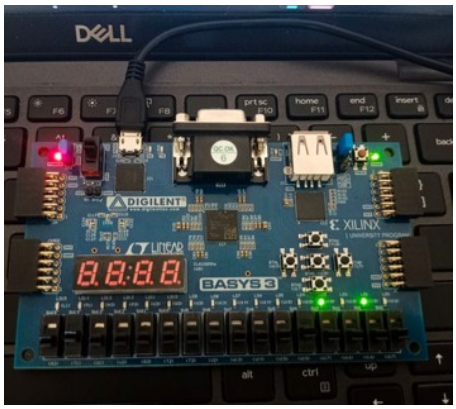
1. Locate SW0 (rightmost switch) = input a, SW1 = input b
2. LEDs from right: LD0=y_and, LD1=y_not, LD2=y_or, LD3=y_xnor, LD4=y_xor
3. Set both switches DOWN (0) for Test Case 1
4. Observe which LEDs light up — compare to truth table
5. Repeat for all 4 switch combinations

10.2 Hardware Test Results

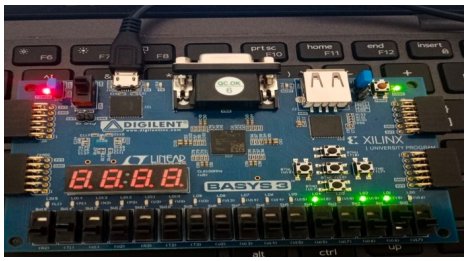
Test Case	SW1 (b)	SW0 (a)	y_and LED	y_or LED	y_not LED	y_xor LED	y_xnor LED	Result
00	↓ OFF	↓ OFF	OFF	OFF	ON	OFF	ON	PASS
01	↑ ON	↓ OFF	OFF	ON	ON	ON	OFF	PASS
10	↓ OFF	↑ ON	OFF	ON	OFF	ON	OFF	PASS
11	↑ ON	↑ ON	ON	ON	OFF	OFF	ON	PASS

10.3 Hardware Photos — All 4 Input Combinations

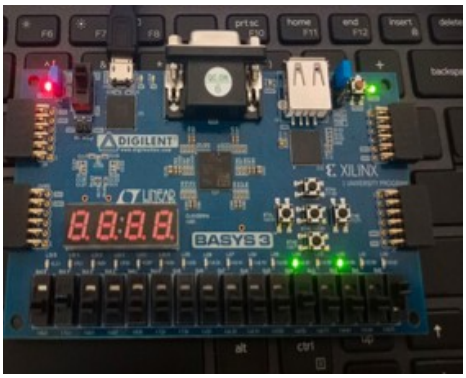
The following photos show the Basys 3 board during actual hardware testing for all four input combinations:



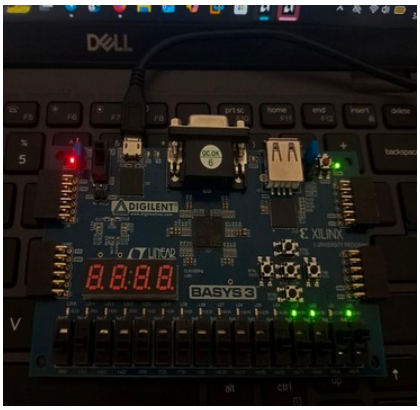
Test Case 00: a=0, b=0 — y_not & y_xnor LEDs ON



Test Case 01: a=0, b=1 — y_or, y_not, y_xor LEDs ON



Test Case 10: a=1, b=0 — y_or, y_xor LEDs ON



Test Case 11: a=1, b=1 — y_and, y_or, y_xnor LEDs ON

All four hardware test cases match the expected truth table. The implementation is verified correct on physical hardware.

11. Complete Vivado Flow Summary

Step	Stage	Action	Output / Artifact
1	Project Setup	Create Vivado RTL project	Logic_Gates.xpr project file
2	RTL Design	Write Logicgates.v	Behavioral Verilog module
3	Testbench	Write tb_Logicgates.tb	Simulation stimulus file
4	Simulation	Run XSim behavioral sim	Waveform — all 4 cases PASS
5	Synthesis	Run Vivado synthesis	Gate-level netlist (LUT1, LUT2)
6	Constraints	Write constraints.xdc	Pin & I/O standard mappings
7	Implementation	Place & Route on Artix-7	Placed netlist, routed design
8	Power Analysis	Review power report	1.63 W total (I/O dominated)
9	Bitstream	Generate .bit file	Logicgates.bit programming file
10	Programming	Program Basys 3 via USB	FPGA configured and running
11	Verification	Test all 4 switch combos	Hardware results match truth table

12. Conclusion

The complete Xilinx Vivado design flow was successfully executed for a 5-gate combinational logic module. Every stage — from RTL coding to physical FPGA verification — was completed and all outputs matched the expected truth table for all four input combinations (00, 01, 10, 11).